

Assignment 1 (10% of total marks)

**Due date: Saturday, 24 January 2026 by 9:00 pm (21:00 hours)
Singapore time.**

Scope

The tasks of this assignment consist of exercises in the areas of **Data Structures and Algorithms**, focusing on algorithm complexity analysis and recursion algorithms. The assignment covers the topics discussed in **Topics 1, 2, and 3**.

The assignment consists of:

- **Part A:** Theory questions to assess your understanding of the subject matter and support the programming question's objective.
- **Part B:** Programming questions requiring the implementation of concepts related to algorithms and their complexity.

Marks for each question are indicated next to the respective question. The total mark for Assignment 1 is **100 marks**.

Marks Distribution

- **Part A:** 60%
- **Part B:** 40%
- **Total Marks:** 100
- **Weightage:** 10% of the total subject mark

Assessment Criteria

Marks will be awarded for:

- **Correctness:** Solutions must meet the requirements of the question.
- **Comprehensiveness:** Solutions should demonstrate a complete understanding of the topic.
- **Appropriate Application:** Answers should reflect proper application of the materials covered in this subject.

Deliverables

Part A: Theory Questions (Questions 1 to 3)

- Type your answers using **MS Word** and save the completed document as a **PDF file**.
- Alternatively, you may write your answers by hand onto a piece of paper or using tablet. Ensure your handwriting is **neat**, then scan or save your handwritten solutions into a **PDF file**.

Part B: Programming Question (Question 4)

- Implement your solution following the standard requirements provided for the question. You may use **C, C++, Java, or Python**.
- Execute your program and take a **screenshot of the output**.
- Save your source code in **pure ASCII text format** with the appropriate file extension (e.g., **.cpp** for C++, **.java** for Java, or **.py** for Python).
- Ensure your program is appropriately documented with comments.
- Provide clear **compilation instructions** in a readme.txt file. Include batch or makefiles if applicable for C or C++ programs.
 - *Note:* If no compilation instructions are provided, your lecturer will attempt compilation based on their environment. Any failure due to incompatibility will result in a "failure to compile" outcome.

Submission Instructions

- Combine the following into a single **ZIP file** named **SolutionA1.zip**:
 - Your solutions to the theory questions (in **PDF format**).
 - Your source code for the programming question (in **ASCII text format**).
 - The screenshot of your program's output.
- Do not include subdirectories in your submission.

Part A: Theory Questions (60 Marks)

Question 1 – Algorithm Complexity Analysis (15 Marks)

a) Arrange the following functions in increasing order; that is, $f(n)$ should come before $g(n)$ in your list if and only if $f(n)$ is $O(g(n))$. Note: $n \geq 0$. Justify all your answers.

- $3^{\log_3(\log_2 n)}$,
- 2^{3n} ,
- $(n - 3)!$,
- $10n^0$,
- $\sqrt{16n^2}$,
- $\log_2(\log_2(n + 10))^n$,
- $12^{\log_2 n^2}$,
- $\sum_{k=1}^n k$,
- $2^{\log_2 \log_2 n} + \sqrt[3]{n}$,
- $\log_2((\log_2 n)^3)$

(5.0 marks)

b) Give a Big-Oh characterization, in terms of n , of the running time of the method `myMethod()`. Count the approximate number of basic operations – assignments, comparisons, and arithmetic operations. Show all details of your answer.

```
def myMethod(int n):  
    total = 0  
    for i = 1 to n:          # inclusive range [1, n]  
        j = 1  
        while j < n:  
            total = total + 1  
            j = j * 2  
    return total
```

(5.0 marks)

c) Prove using the formal definition. Find constants c and n_0 .

- i. Prove: $7n^3 - 2n^2 + 8 \in \Theta(n^3)$ **(1.0 mark)**
- ii. Prove: $n^2 + 100n \in \Omega(n^2)$ **(1.0 mark)**
- iii. Prove: $\sum_{k=1}^n k \in \Theta(n^2)$ **(1.0 mark)**
- iv. Prove: $n \log_2 n \in \Omega(n)$ **(1.0 mark)**
- v. Prove: $3n^2 + 7n + 20 \in O(n^2)$ **(1.0 mark)**

Question 2 – Recursive Algorithm (15 Marks)

Your lecturer is a funny guy; given a **sorted** list of numbers, he will use the following recursive search algorithm to search the smallest number:

```
static int minimum ( L, left, right)
if (left == right) then
    return L[left]
else
    temp1 ← minimum(L, left, (left + right)/2)
    temp2 ← minimum(L, ((left + right)/2) + 1, right)
    if temp1 ≤ temp2
        return temp1
    else return temp2
```

(a) Analyse the worst-case asymptotic complexity of the algorithm your lecturer uses. Give the worst-case running time in terms of θ notation. Explain your answer.

(10.0 marks)

(b) How does this algorithm compare to the algorithm that simply linearly iterates through the list to look for the target? Note, you need to compare both under the worst-case running time. Explain your answer. (Hint: Tricky, do not get mis-led by the question.)

(5.0 marks)

Question 3 – Master Theorem (30 Marks)

For each of the following recurrence relations, determine the runtime $T(n)$ complexity. Use the Master Theorem to solve the recurrence relations if the recurrence relations can be solved using Master Theorem, otherwise use expansion (or recursive substitution) to solve the recurrence relations. You must show all your working to justify your answer. Solution without clear justification or steps scores no mark.

a) $T(n) = 16T\left(\frac{n}{4}\right) + n^4$, and $T(1) = 1$ (6.0 marks)

b) $T(n) = 4T\left(\frac{n}{2}\right) + \frac{n^2}{\log_2 n}$, and $T(1) = 1$ (6.0 marks)

c) $T(n) = 4T\left(\frac{n}{2}\right) + n^2\sqrt{n}$, and $T(1) = 1$ (6.0 marks)

d) $T(n) = 4T\left(\frac{n}{4}\right) + n\log_2^2 n$, and $T(1) = 1$ (6.0 marks)

e) $T(n) = T(n - 1) + n^2$, and $T(1) = 1$. (6.0 marks)

Part B: Programming Question (40 Marks)

Question 4 (40 Marks)

- a) In a recent UOW-SIM orientation camp, there are m number of male students and f number of female students. Like all previous orientation camps, the number of male students, of different ages, is higher than the number of female students; that is $m > f$. The organizer of the camp would like to find out the ages among the students attending the camp that are common between both the male and female students. Device an efficient algorithm that runs on $O((m + f) \log_2 m)$ complexity, to achieve the needs of the organizer, that is, describe an algorithm (in pseudocode) that runs on $O((m + f) \log_2 m)$ to find out the ages that are common between both the male and female students. You may use the following two lists as reference when you design your algorithm.

For example:

Age_{male}:

21	19	24	22	20	23	18
----	----	----	----	----	----	----

Age_{female}:

17	17	22	19
----	----	----	----

Then, the output of your program should be 22, 19.

(15.0 marks)

- b) Write a Java, C++, or Python program that implements the algorithm that you have come out with in Part a. You may use existing codes for quicksort algorithm or binary-search algorithm that meet the specified criteria (that is, $m \log m$ and $\log m$ run-time complexity respectively). Alternatively, you can write your own quicksort and binary-search algorithms. **(20.0 marks)**
- c) Analyze the running time complexity of your program, and include the analysis report as part of the deliverable for Part b. **(5.0 marks)**

Notes on Submission

1. This assignment is due on **Saturday, 24 January 2026 by 9:00 pm (21:00 hours) Singapore time.**
2. Submission must be made via **Moodle**. Ensure all files are combined into a single **ZIP file** without subdirectories.
3. Clearly label all components of your submission.
4. Late submissions will incur a **5% deduction per day**, including weekends.
5. Submissions more than **seven** days late will not be marked unless an extension is granted.
6. Extensions must be applied for via **SOLS** before the assignment deadline.
7. Plagiarism is treated seriously. Instances of plagiarism may result in a zero mark for the assignment.

Submit the file **SolutionA1.zip** through Moodle in the following way:

- 1) Access Moodle at **<http://moodle.uowplatform.edu.au/>**
- 2) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
- 3) When successfully logged in, select a site **CSCI203 (SP126) Algorithms and Data Structures**
- 4) Scroll down to a section **Submissions of Assignments**
- 5) Click at **Submit** your Assignment 1 here link.
- 6) Click at a button **Add Submission**
- 7) Move the files created into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add...*
- 8) Click at a button **Save** changes,
- 9) Click at check box to confirm authorship of a submission,
- 10) When you are satisfied, remember to click at a button **Submit assignment**.

A policy regarding late submissions is included in the subject outline. Only one submission per student is accepted.

Assignment 1 is an individual assignment, and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification

If you have any questions or require clarification, please contact your lecturer as early as possible. Good luck!